A3-PS020-TC172

Local SRv6 Path Steering and Measured Failure Recovery

Category: 6G & Future Networks | Problem Statement PS-020

Team NetNova

Shivani R M • Divya Sree N

Mentor: Dr. Shweta Babu Prasad

shivanirm26482@gmail.com

RV College of Engineering

AIORI Hackathon 3

October 13, 2026

Abstract

Problem Description

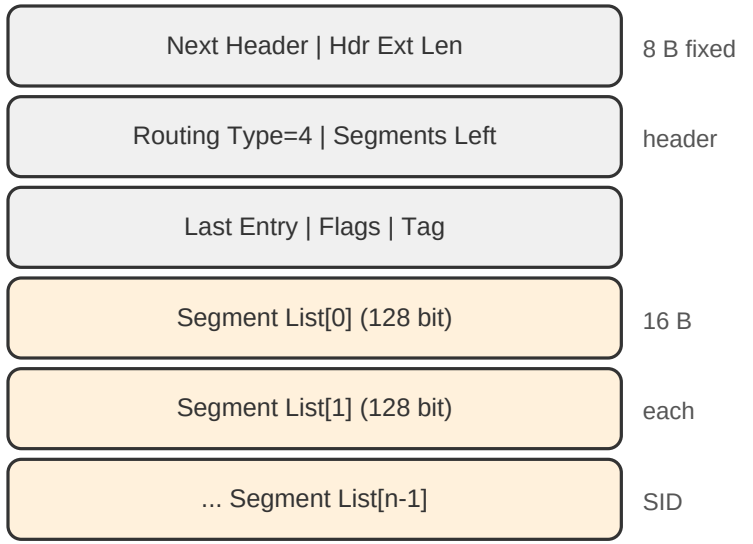
- In normal IPv6, every router chooses the next hop on its own. We cannot choose the path, and after a link failure traffic stays broken until the routing protocol notices and recomputes.
- SRv6 (RFC 8754, RFC 8986) lets the first router put a list of SIDs in the packet header, so the packet follows a path we picked.
- We want to find out how SRv6 steering behaves compared with plain IPv6 routing when the same link fails in the same topology, and how many extra header bytes it costs.
- We build a virtual lab with 5 routers, configure 2 explicit SRv6 paths, fail one link, and run plain IPv6 (OSPFv3) as the baseline on the same topology.
- We measure packet loss, interruption time, failure detection delay, repair delay and SRv6 header overhead. Each failure test is repeated at least 30 times, and packet captures confirm the SID path.
- What is different in our approach: we split recovery time into detection and repair, we compare three modes (plain IPv6, SRv6 with a scripted backup switch, SRv6 with the backup path already installed), and we package the lab so the mentor can re-run it.
- As the problem statement says, we do not claim SRv6 is always better and we do not study SID summarisation. TI-LFA/BFD is only a stretch goal.

Background

What you need to know to follow the experiment

- IPv6 routing is hop by hop. After a failure the network has to detect it, flood the change, rerun the shortest path calculation and update the forwarding table [5].
- With SRv6 the first router adds a Segment Routing Header (SRH) holding SIDs, which are 128-bit addresses. A SID names a node and an action: End moves to the next SID, End.X crosses a chosen link, End.DT6 removes the outer header and looks up the inner destination [3][2].
- Header cost is easy to calculate: the SRH is 8 bytes plus 16 bytes per SID [1]. With H.Encaps there is also a 40-byte outer IPv6 header. The reduced version, H.Encaps.Red, leaves the first SID out of the SRH and saves 16 bytes [3].
- Why recovery time matters for 6G: future networks need very reliable, low-latency transport. Carrier networks have long used 50 ms as the target for protection switching.
- Linux supports SRv6 in the kernel (seg6) [8], so the whole lab can run on one machine using network namespaces and FRRouting.

SRH layout (RFC 8754)



Total = 8 + 16n bytes. Example: n = 2 SIDs gives 40 B SRH, plus 40 B outer IPv6 under H.Encaps = 80 B.

Literature Review

Related work by theme: what each source gives us, what it leaves open, and why it matters for PS-020

Theme	Source	Contribution	Limitation (what it leaves open)	Relevance to PS-020
SRv6 architecture & data plane	**[1][3] RFC 8754, RFC 8986** Filsfils et al., 2020-21	Define the SRH, SID format and behaviours (End, End.X, End.DT6, H.Encaps.Red).	Specify the mechanism only; no recovery times and no measured overhead.	**Basis of our SID lists, the 8 + 16n byte cost, and the capture checks.**
	[2][4] RFC 8402, RFC 9256 Filsfils et al., 2018 / 2022	SR architecture; SR Policy with candidate paths, preference and binding SID.	State sits at the ingress; recovery behaviour is not quantified.	**Primary and backup policy define modes M1 and M2; binding SID holds pre-staged P2.**
IP routing convergence (baseline)	**[5] RFC 5340** Coltun et al., 2008	OSPFv3, the link-state protocol that converges plain IPv6 routing.	Convergence depends on timers and topology; no SRv6 comparison.	**Baseline mode M0 on the same topology and the same failure.**
	[11] Francois et al. SIGCOMM CCR, 2005	Shows IGP convergence can reach sub-second values and breaks it into stages.	Plain IGP only; no explicit-path alternative to compare with.	**Model for our T_detect / T_repair split of recovery time.**
	[13] Markopoulou et al. IEEE/ACM ToN, 2008	Characterises failures in an operational IP backbone; many are short-lived, single-link events.	Gives failure statistics, not a recovery mechanism or its cost.	**Justifies a single-link failure (R2-R5) as the test case.**
Failure detection & fast protection	**[6] RFC 5880 (BFD)** Katz, Ward, 2010	Fast liveness detection between adjacent systems.	Covers detection time only, not path steering.	**Detector for the silent failure test (stretch goal).**
	[7][12] RFC 5286 (LFA), RFC 9855 (TI-LFA) Atlas, Zinin, 2008 / Bashandy et al., 2025	Precomputed repair paths; TI-LFA encodes them as SID lists for any two-connected topology.	Specification level; no open, repeatable comparison with plain OSPFv3 recovery.	**Idea behind the pre-installed backup in M2; full TI-LFA only as stretch.**
Implementation & measurement	**[8] Lebrun, Bonaventure** ANRW, 2017	SRv6 support in the Linux kernel (seg6).	Describes the implementation; no failure experiments.	**Our emulation platform; we state its limits.**
	[9] Abdelsalam et al. IEEE TNSM, 2021	SRPerf: automated, repeatable SRv6 performance evaluation.	Measures forwarding performance, not failure recovery.	**Method reference for scripted, repeatable runs.**
	[10] Ventre et al. IEEE COMST, 2021	Survey of SR research, standardisation and implementations.	Broad overview; no single controlled recovery test.	**Places PS-020 in the field and shows the gap we address.**

Sources: RFC 9855 (TI-LFA) published October 2025. Full references on the Bibliography slide.

Research Gap & Novelty

What the reviewed literature does not show together, and what PS-020 adds

Source	SRv6 explicit-path steering	Compared with plain IPv6 under same failure	Failure recovery measured	Detection vs repair split	Pre-installed backup path	Header overhead quantified	Re-runnable lab package
[1][3] RFC 8754, 8986	●	○	○	○	○	◐	○
[2][4] RFC 8402, 9256	●	○	○	○	◐	○	○
[5][11] OSPFv3, Francois	○	○	●	●	○	○	○
[6] BFD	○	○	○	◐	○	○	○
[7][12] LFA, TI-LFA	◐	○	○	○	●	◐	○
[8] Lebrun, Bonaventure	●	○	○	○	○	○	○
[9] SRPerf	●	○	○	○	○	○	◐
[10] Ventre survey	●	○	◐	○	○	○	○
PS-020 (this work)	●	●	●	●	●	●	●

● addressed ◐ partly ○ not addressed

Ratings follow each source's stated scope; they are our reading of the literature, not a benchmark.

Research questions

RQ1 After the same link failure, how do loss and interruption time compare for plain IPv6 (OSPFv3), SRv6 with a scripted switch and SRv6 with a pre-installed backup?

RQ2 How is recovery time split between detection and repair, for loud and silent failures?

RQ3 What header bytes does each SRv6 path cost, and how do they sit next to the recovery gain?

Novelty of PS-020

N1 One controlled comparison of three modes on a single topology, failure instant and clock.

N2 Recovery split into T_detect and T_repair; N >= 30 randomised runs with 95% CI.

N3 Recovery time reported together with SRH overhead per path.

N4 Open, scripted lab (pcaps, make demo, analysis notebook) the mentor can re-run.

Scope of claim To our knowledge, none of the sources reviewed here compares plain IPv6, SRv6 with a scripted switch and SRv6 with a pre-installed backup under one failure. We do not claim SRv6 is always better, and we do not study SID summarisation.

Proposed Solution

A controlled lab where recovery time is measured part by part

1. Testbed

- Linux network namespaces, kernel SRv6 (seg6) + FRRouting.
- Pentagon ring R1-R2-R5-R4-R3-R1; source S behind R1, sink D behind R5.
- Locators fc00:0:N::/48. SIDs: End = fc00:0:N::1, End.DT6 on R5 = fc00:0:5::d6.
- Path P1 (primary): R1-R2-R5, SID list {2::1, 5::d6}.
- Path P2 (backup): R1-R3-R4-R5, SID list {3::1, 4::1, 5::d6}.
- Link under test: R2-R5.

2. Three compared modes

- M0 Baseline: plain IPv6 + OSPFv3, hop-by-hop, no SRH.
- M1 SRv6 with scripted switch: traffic uses P1; after a failure a script replaces the policy with P2.
- M2 SRv6 with backup ready: P2 is already installed behind a binding SID, so a failure only changes the preference.
- All modes share the identical topology, link costs, traffic and failure instant.
- SID traversal checked with tcpdump on every link: destination address changes and Segments Left counts down to 0.

3. Failure and measurement

- Traffic: UDP probes, 1000 pkt/s, sequence number + send timestamp.
- Failure A (loud): ip link set down on R2-R5.
- Failure B (silent): netem 100% loss, found by liveness probes or BFD (stretch).
- Metrics: packet loss, interruption, T_detect, T_repair, header overhead.
- All routers share one kernel clock, so no clock sync error.
- N >= 30 runs per mode; mean, median, p95, 95% CI.

Solution Optimization

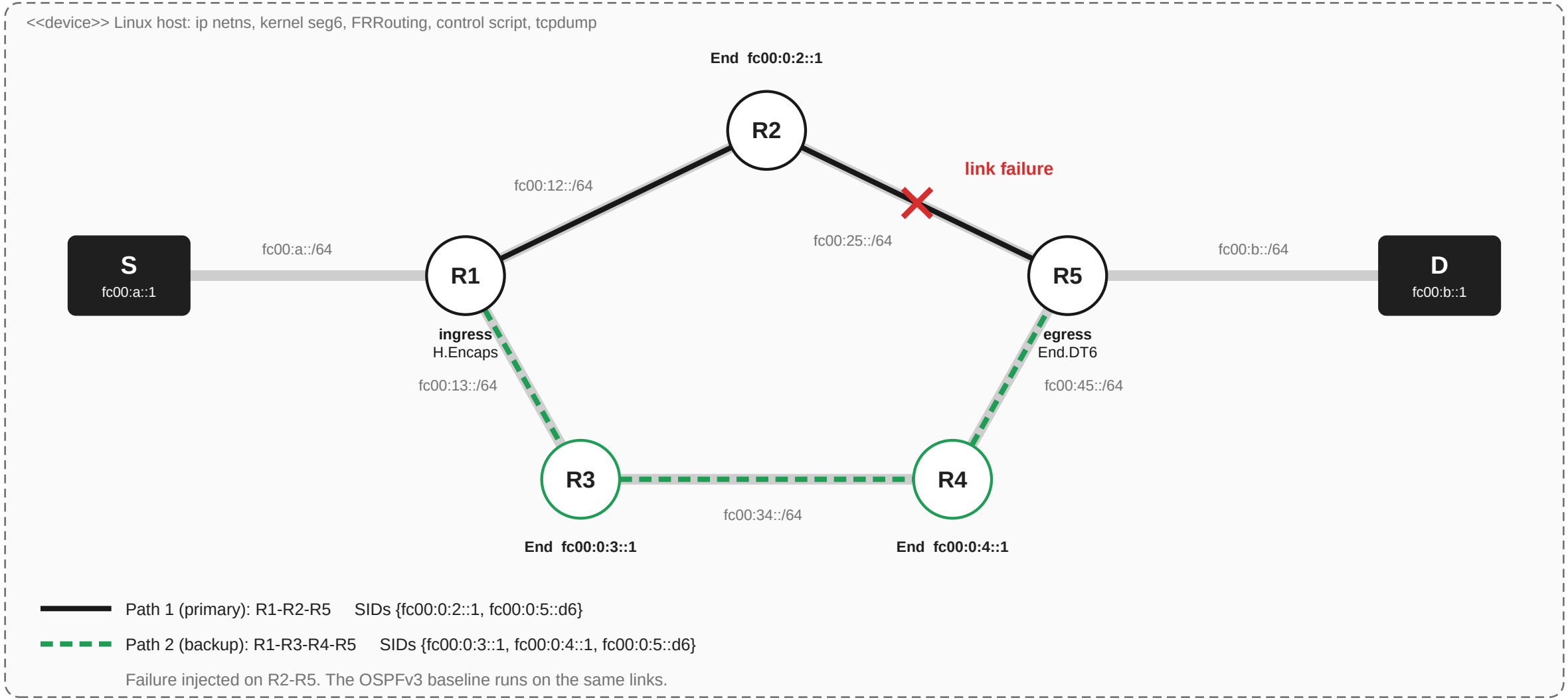
What we change from the naive version (effects are expected, we will confirm by measuring)

	Naive version	What we do	Expected effect
Policy switch	Delete the old route, then add the new one. Packets drop in between.	One atomic `ip -6 route replace`. No gap.	**Less loss during repair**
Backup path	Work out and install it after the failure.	Install P2 beforehand behind a binding SID. A failure only flips the preference.	**Shorter repair delay**
Detecting failure	Poll every 100 ms.	Netlink link events for the loud case. 50 ms probes x 3 (or BFD) for the silent case.	**Detection time is bounded and measured**
Header size	Full H.Encaps: 40 B outer header + 8 + 16n.	H.Encaps.Red if the kernel allows it. Saves 16 B per packet.	**Lower overhead**
Measuring	ping once a second, so 1000 ms resolution.	1000 probes/s with sequence numbers, so 1 ms resolution.	**Interruption time is accurate**
Statistics	One run per failure.	30 or more runs, random failure offset, 95% CI.	**Comparison we can trust**
Reproducing	Type the commands by hand.	Scripted topology, `make demo`, raw pcaps, analysis notebook.	**Mentor can re-run it**

Recovery time = T_detect + T_decide + T_repair + T_propagate Header cost: O = 40 + 8 + 16n bytes (H.Encaps)

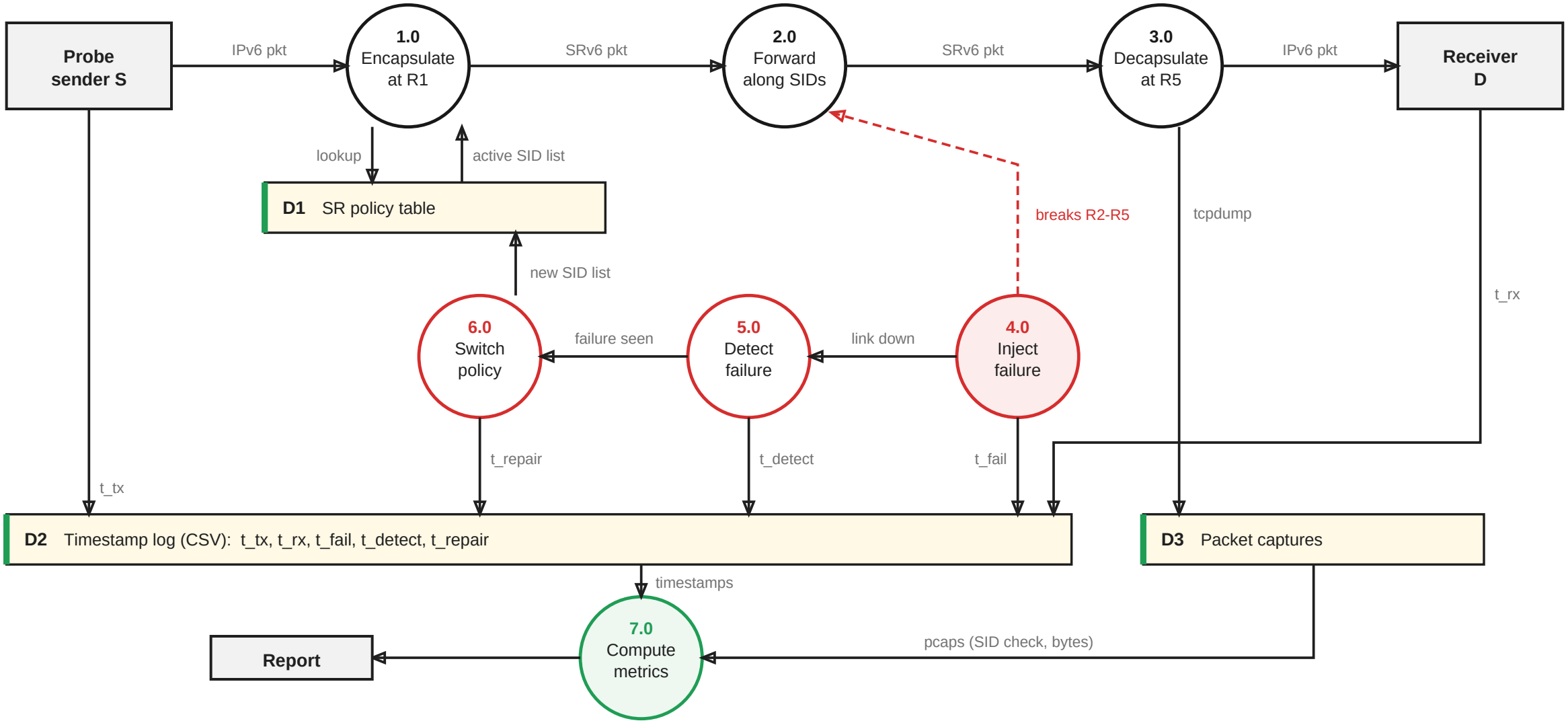
Solution Architecture & Design

Deployment view: five routers as Linux network namespaces on one host



Data Flow Diagram

Level-1 DFD: probe traffic on top, failure and recovery in the middle, measurement at the bottom



Implementation Status

What is built, what is recorded, and what is only planned

Implemented (built) — BUILT

- Linux network-namespace testbed: R1–R5 plus source and destination namespaces (Ubuntu, iproute2).
- IPv6 addressing and connectivity; FRRouting OSPFv3 baseline (M0).
- SRv6 (kernel seg6): SIDs, End and End.DT6 behaviours, SRv6 encapsulation routes.
- Primary path R1→R2→R5 and backup path R1→R3→R4→R5; forwarding verified.
- Probe-based recovery controller: 3 consecutive failed probes → install backup SRv6 path; timestamps logged; connectivity re-verified.
- Modes M0, M1, M2; tcpdump captures; Python / Bash automation and analysis.
- React + Vite read-only dashboard, deployed on Render.

Recorded (measured outputs) — RECORDED

- Packet loss, mean RTT, maximum RTT.
- Failure detection delay, repair delay, total recovery time.
- SRv6 header overhead.
- Repeated trials with statistical analysis.
- Saved experiment results are shown in the dashboard (Benchmark Analytics, Experiment Logs).
- **Numeric values are not reproduced in this addendum.** Quote them only from the raw logs in the repository *evidence* folder, so slides and data always agree.

Not implemented / future — FUTURE

- Automatic failback: the controller switches one way only (primary → backup).
- Live control from the dashboard: it is read-only and visualises saved results.
- TI-LFA-style repair and BFD-based detection: stretch goals in the proposal, not claimed as done.
- Any claim that SRv6 is always faster than OSPFv3: results apply to this topology, this failure and these timers only.

Source of this slide: team project description and repository README. The repository *src*, *backend* and *evidence* folders were not machine-readable when this addendum was prepared, so items marked Recorded must be checked against them before presenting.

Linux Implementation

One Ubuntu host, five router namespaces, kernel SRv6

Environment and tools

- **Ubuntu Linux**: single host; each router (R1–R5) and the source and destination hosts is a **network namespace**.
- **iproute2**: namespaces, links, IPv6 addresses, SRv6 SIDs and encapsulation routes.
- **Kernel seg6**: SRH insertion, End and End.DT6 processing.
- **FRRouting**: OSPFv3 for the plain IPv6 baseline.
- **tcpdump**: captures to confirm SID traversal and header bytes.
- **Python and Bash**: topology setup, probes, controller, metrics and analysis.
- All namespaces share one kernel, hence one clock for every timestamp.

Implementation workflow

1. Set up the Ubuntu environment.
2. Create namespaces for the five routers.
3. Assign IPv6 addresses; configure connectivity.
4. Configure the OSPFv3 baseline.
5. Configure SRv6 SIDs.
6. Configure policy-based path steering.
7. Simulate the link failure; switch to the backup path.
8. Capture packets, measure recovery, analyse performance.

Topology (from the proposal): pentagon ring R1–R2–R5–R4–R3–R1, source behind R1, destination behind R5; link under test R2–R5; locators fc00:0:N::/48, End = fc00:0:N::1, End.DT6 on R5 = fc00:0:5::d6.

SRv6 Path Configuration

Two explicit paths, three routing modes

SID lists (design values from the proposal)

- P1 primary: R1→R2→R5, SID list {fc00:0:2::1 (End), fc00:0:5::d6 (End.DT6)}
- P2 backup: R1→R3→R4→R5, SID list {fc00:0:3::1, fc00:0:4::1, fc00:0:5::d6}
- R1 is the ingress and encapsulates; R5 is the egress and decapsulates (End.DT6).
- Verification: tcpdump on each link; destination address changes per hop and Segments Left counts down to 0.

Header overhead (calculated, not measured)

- SRH = 8 + 16n bytes; H.Encaps adds a 40 B outer IPv6 header.
- P1, n = 2: SRH 40 B, total 80 B.
- P2, n = 3: SRH 56 B, total 96 B.
- P2 costs 16 B more per packet than P1. Compare with the capture-based figure in the evidence folder.

Routing modes compared (same topology, traffic and failure)

- **M0** plain IPv6 + OSPFv3, hop-by-hop, no SRH; recovery depends on OSPFv3 detection, flooding and SPF.
- **M1** SRv6 on P1; after the controller detects the failure, the backup SRv6 path is installed by script.
- **M2** SRv6 with the backup path pre-installed; a failure only changes which path is used, so repair delay should be shorter (to be confirmed by recorded data).

The SID values and path lists above are the design in the submitted proposal. Check them against the scripts in *src* before presenting.

Failure Detection and Recovery

Probe-based controller, one-way switch to the backup path

Controller logic (implemented)

- Send periodic probes towards the destination to monitor reachability.
- Declare failure after **three consecutive failed probes**; record t_detect.
- Install the backup SRv6 path (P2) at R1; record t_repair.
- Verify connectivity through the backup path.
- Log timestamps and compute packet loss, RTT, detection delay, repair delay and total recovery time.
- **Limitation:** one-way switching only. Automatic failback to P1 is not implemented, so the network stays on P2 after recovery of the link.

Metrics and how they relate

- Detection delay = t_detect – t_fail.
- Repair delay = t_repair – t_detect.
- Total recovery time = detection delay + repair delay (traffic restored on the backup path).
- Packet loss and mean / maximum RTT come from the probe stream across the run.
- Because detection needs three consecutive failed probes, **detection delay is bounded below by the probe interval**, so the interval used must be stated next to every result.
- Repeated trials: report mean, median, p95 and 95% confidence interval from the raw logs.

Proposal vs build: the proposal also describes netlink link events for loud failures, a silent-failure test (netem) and BFD. Describe only those the repository scripts actually use; the controller described here is probe-based.

Dashboard and Deployment

Read-only visualisation of saved results

SRv6 Network Operations Dashboard

- Built with React and Vite; deployed on Render.
- Modules: Overview, Network Topology, Routing Modes, SRv6 Path Steering, Failover Monitoring, Benchmark Analytics, Experiment Logs, Settings.
- Shows **saved** experiment results and configuration.
- It is **not** a live network controller: it does not run experiments or change routes.

Resources

- Live dashboard: a3-ps020-tc172-5.onrender.com
- Repository: github.com/aiori-hackathon/A3-PS020-TC172
- Team introduction: youtube.com/shorts/kKTfiE7RIRU
- Implementation and live demo: youtu.be/PvIQ_SrlY2A
- Repository folders: backend, evidence, frontend, src.

Evaluation tip: when asked about results, answer with the evidence folder and the dashboard figures, and state the limits (emulation on one host, probe interval, one-way switching).

Deployment Timeline

Milestones and method; each phase ends with a check, and durations fit the hackathon window

Phase	Work	Exit criterion (verified by)	Share
P1 Testbed	5 namespaces, links, addressing, FRR OSPFv3, S and D hosts	S pings D on every IPv6 path; OSPFv3 converged	15%
P2 Baseline M0	Plain IPv6 reference, probe sender/receiver, logging	Zero loss at steady state; probe resolution 1 ms	10%
P3 SRv6 paths	P1 and P2 SID lists, End / End.DT6, SR policy at R1	Captures show the SID path and Segments Left counting down to 0	20%
P4 Failure + recovery	Failure injector, detector, atomic policy switch, pre-staged M2	Interruption measured end to end for M0, M1 and M2	20%
P5 Experiments	30+ randomized runs per mode, loud and silent failure, overhead capture	Raw CSV and pcaps archived in GitHub	15%
P6 Analysis + stretch	Statistics, plots, BFD / TI-LFA-style stretch if time permits	Comparison tables and CDFs reproducible by script	10%
P7 Delivery	Technical report, pseudocode snippet, live demo to mentor	Private repo shared with aiori-hackathon; make demo works	10%

Method: one shared clock, same topology and traffic in every mode, random failure timing, all results regenerated by script from raw logs.

Bibliography

[1] C. Filsfils et al. "IPv6 Segment Routing Header (SRH)." RFC 8754, IETF, March 2020.

[2] C. Filsfils et al. "Segment Routing Architecture." RFC 8402, IETF, July 2018.

[3] C. Filsfils et al. "Segment Routing over IPv6 (SRv6) Network Programming." RFC 8986, IETF, February 2021.

[4] C. Filsfils et al. "Segment Routing Policy Architecture." RFC 9256, IETF, July 2022.

[5] R. Coltun, D. Ferguson, J. Moy, A. Lindem. "OSPF for IPv6." RFC 5340, IETF, July 2008.

[6] D. Katz, D. Ward. "Bidirectional Forwarding Detection (BFD)." RFC 5880, IETF, June 2010.

[7] A. Atlas, A. Zinin. "Basic Specification for IP Fast Reroute: Loop-Free Alternates." RFC 5286, IETF, September 2008.

[8] D. Lebrun, O. Bonaventure. "Implementing IPv6 Segment Routing in the Linux Kernel." Applied Networking Research Workshop (ANRW), 2017.

[9] A. Abdelsalam et al. "SRPerf: A Performance Evaluation Framework for IPv6 Segment Routing." IEEE Trans. Network and Service Management, 2021.

[10] P. L. Ventre et al. "Segment Routing: A Comprehensive Survey of Research Activities, Standardization Efforts, and Implementation Results." IEEE Communications Surveys & Tutorials, 2021.

[11] P. Francois, C. Filsfils, J. Evans, O. Bonaventure. "Achieving Sub-Second IGP Convergence in Large IP Networks." ACM SIGCOMM Computer Communication Review, 2005.

[12] A. Bashandy et al. "Topology Independent Fast Reroute Using Segment Routing." RFC 9855, IETF, October 2025.

[13] A. Markopoulou et al. "Characterization of Failures in an Operational IP Backbone Network." IEEE/ACM Trans. Networking, 2008.

Thank You for Your Attention!

Team NetNova | RV College of Engineering

PS-020: Local SRv6 Path Steering and Measured Failure Recovery